a bit clearer

C PROGRAMMING FROM SCRATCH · EPISODE 14

Variables and Constants

Values that change, and values that don't

KEY POINTS

- A variable needs a type and a name. Names use letters, digits and underscores, and never start with a digit.
- Initialization gives a variable its first value when it's created; assignment gives it a new value later.
- Local variables live in braces, on the stack, and start with garbage. Globals work everywhere, live in the static area and start at 0.
- Three kinds of constants: literals (37, 037, 0×1F, 3.14f, 1.2e3), #define names, and const variables.

01 Naming a variable

▶ VIDEO 00:51

Write a type, a space, then a name: `int age;`

RULE	EXAMPLE
Letters, digits and underscores only	`score`, `_2b`
Never start with a digit	`2b` ×
Capitals count as different	`age` and `Age` are two names
Never a keyword	`int char;` ×
Compilers only have to compare the first 63 characters	keep names short

02 Initialization and assignment

▶ VIDEO 03:20

init-assign.c — C SOURCE

```c
int main()
{
    char letter = 'x';
    int num = 100;
    float score = 95.5f;
    double math = 85.5;

    math = 90.5;
    return 0;
}
```

KEY IDEA

Initialization happens once, when the variable is created (`double math = 85.5;`). Assignment can happen any time after (`math = 90.5;`).

03 Local and global variables

▶ VIDEO 03:30

	LOCAL	GLOBAL
Created	inside braces `{ }`	outside every function
Scope	its own block	the whole file after its definition, even other files
Lifetime	from entering its block to leaving it	the whole run of the program
Lives in	the stack	the static area (data segment)
Starts as	garbage, so always initialize it	`0`

WATCH OUT

`error C4700: uninitialized local variable 'num' used`. Another compiler may only warn and happen to print 0: that's luck. Start with `0`.

When a local and a global share a name, **the local one wins**: with a global `num = 100` and a local `num = 200`, printing `num` inside `main` prints `200`.

Use globals sparingly: functions that share one can't work alone (high coupling). Aim for low coupling, high cohesion.

04 Where variables live

▶ VIDEO 07:01

`int num = 100;` reserves 4 bytes named `num`: the type decides the size. A global is created once, so zeroing it is cheap; a local may be created on every call, so its first value is up to you.

05 Literal constants

▶ VIDEO 08:12

A literal is a value written directly in the code. A leading `0` means octal, `0x` means hex (digits 0–9, then A–F for 10–15), and a suffix sets the type.

LITERAL	MEANING	LITERAL	TYPE
`37`	decimal 37	`120U`	unsigned int
`037`	octal: 3×8 + 7 = **31**	`120L` / `120LL`	long / long long
`0x1F`	hex: 1×16 + 15 = **31**	`120UL`	unsigned long
`1.2e3`	1.2×10^3 = **1200**	`3.14` / `3.14f` / `3.14L`	double / float / long double

06 #define constants

▶ VIDEO 11:05

define.c — C SOURCE

```c
#include <stdio.h>

#define M 100
#define PI 3.14159

int main()
{
    int max = M;
    printf("M = %d\n", M);
    printf("max = %d\n", max);
    printf("PI = %f\n", PI);
    return 0;
}
```

`M` is just a name for `100`, so `M = 200;` fails: `error C2106: '=': left operand must be l-value`.

07 const variables

▶ VIDEO 11:25

`const` makes a variable read-only. It still has a type and an address, but the compiler won't let anyone change it. Assigning to `const int num` after initialization fails:

Build output

```
mistakes\const-assign.c(9): error C2166: l-value specifies const object
```

Use `const` for values that should never change: `const int days = 7;`

CODE 10 files on GitHub

github.com/abitclearer/c-programming-from-scratch/tree/main/14-variables-and-constants